TRABAJO PRÁCTICO OBLIGATORIO

Paradigma Orientado a Objetos — Java

ETAPA 1 · Modelado y aplicación de patrones

Propuesta B: Agencia de Turismo

Sistema propuesto: Rumbo Sur — gestión de tours, salidas, reservas, pagos y operación turística.

Integrante	Ayala Ocampo Alan Daniel
Materia	Paradigma Orientado a Objetos en Java
Alcance	Etapa 1: análisis, casos de uso, diagramas UML, SOLID y GRASP.
Material base	Consigna TPO y apuntes de UM, Patrones GRASP y principios SOLID proporcionados por la docente.

Diagrama de casos de uso

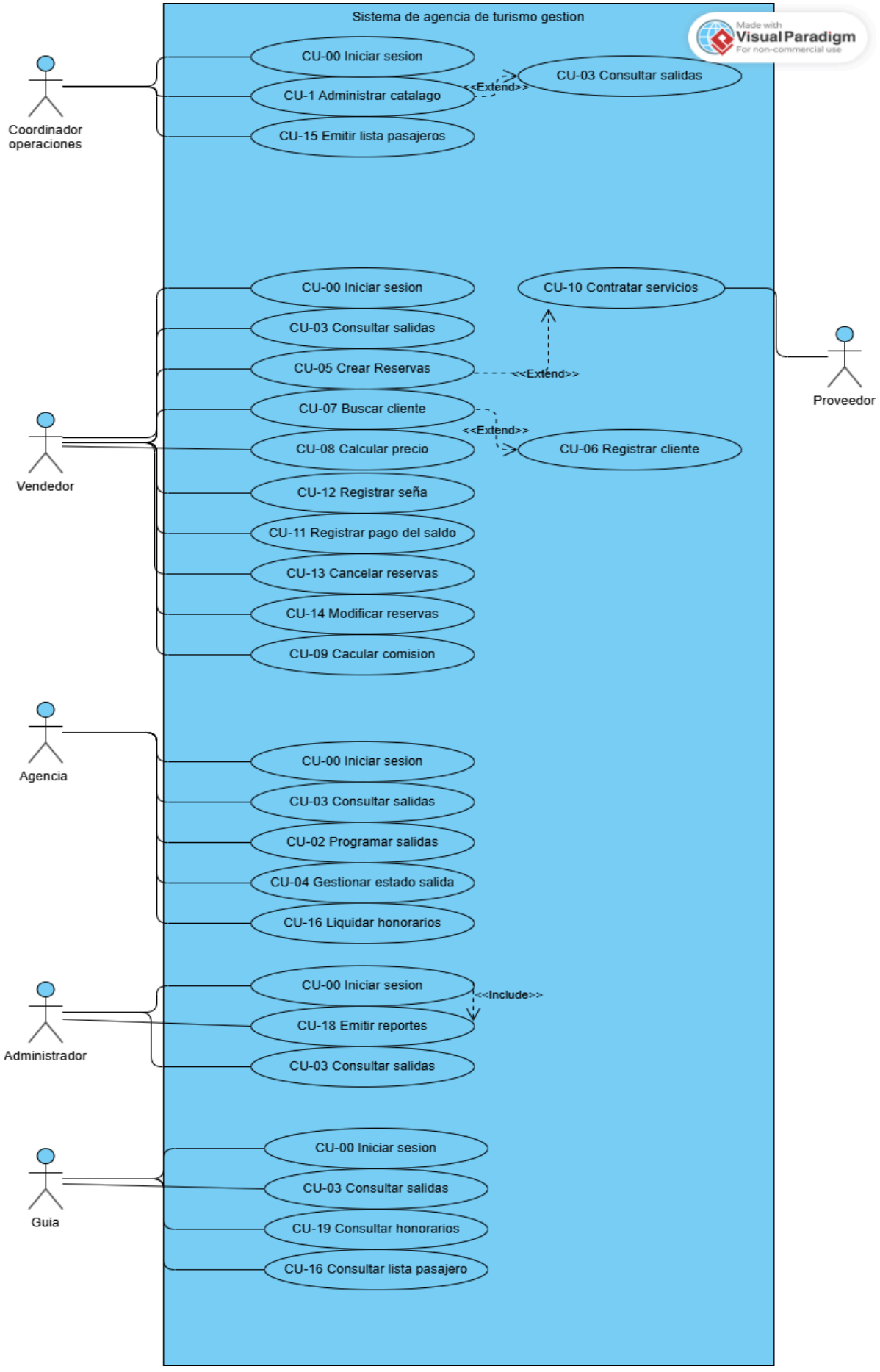
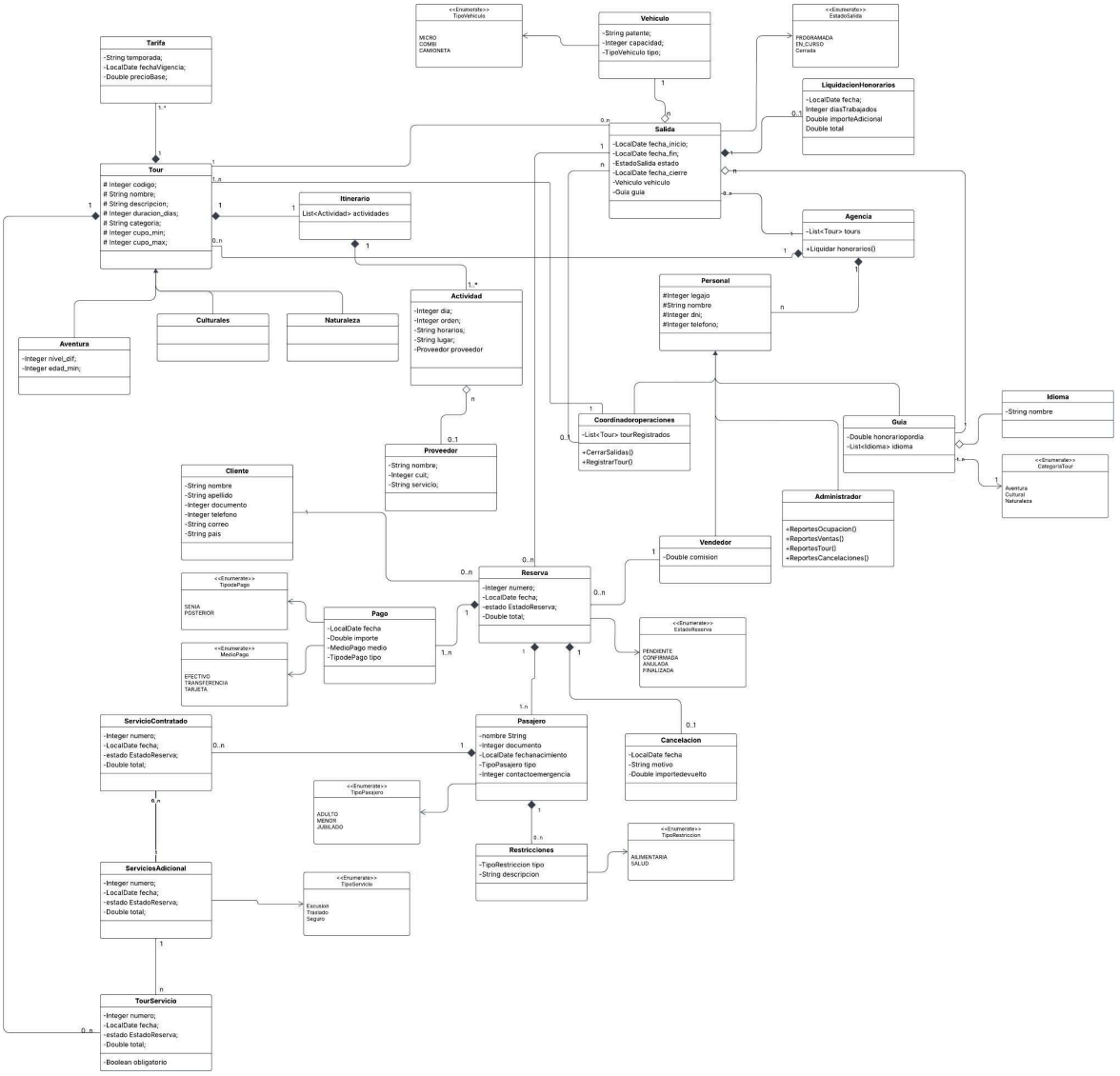
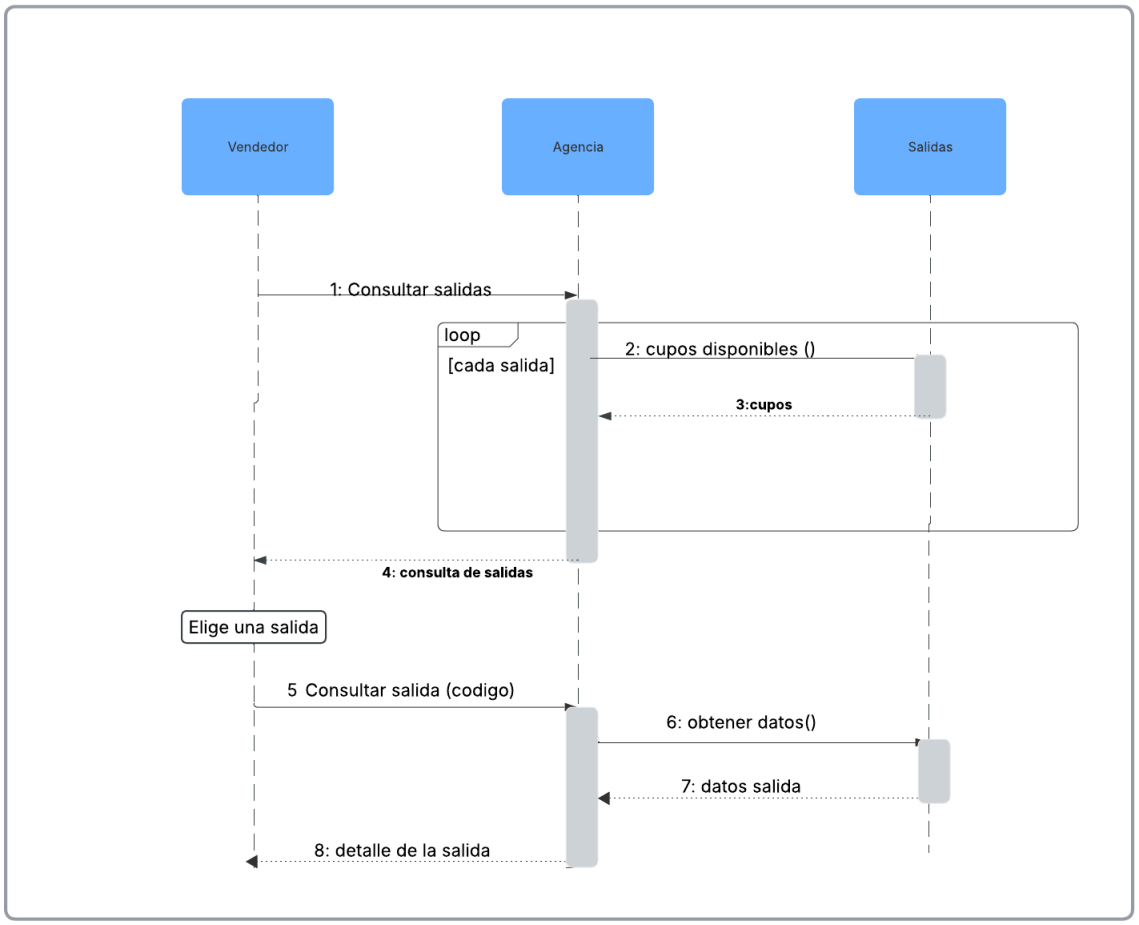


Diagrama de clases inicial



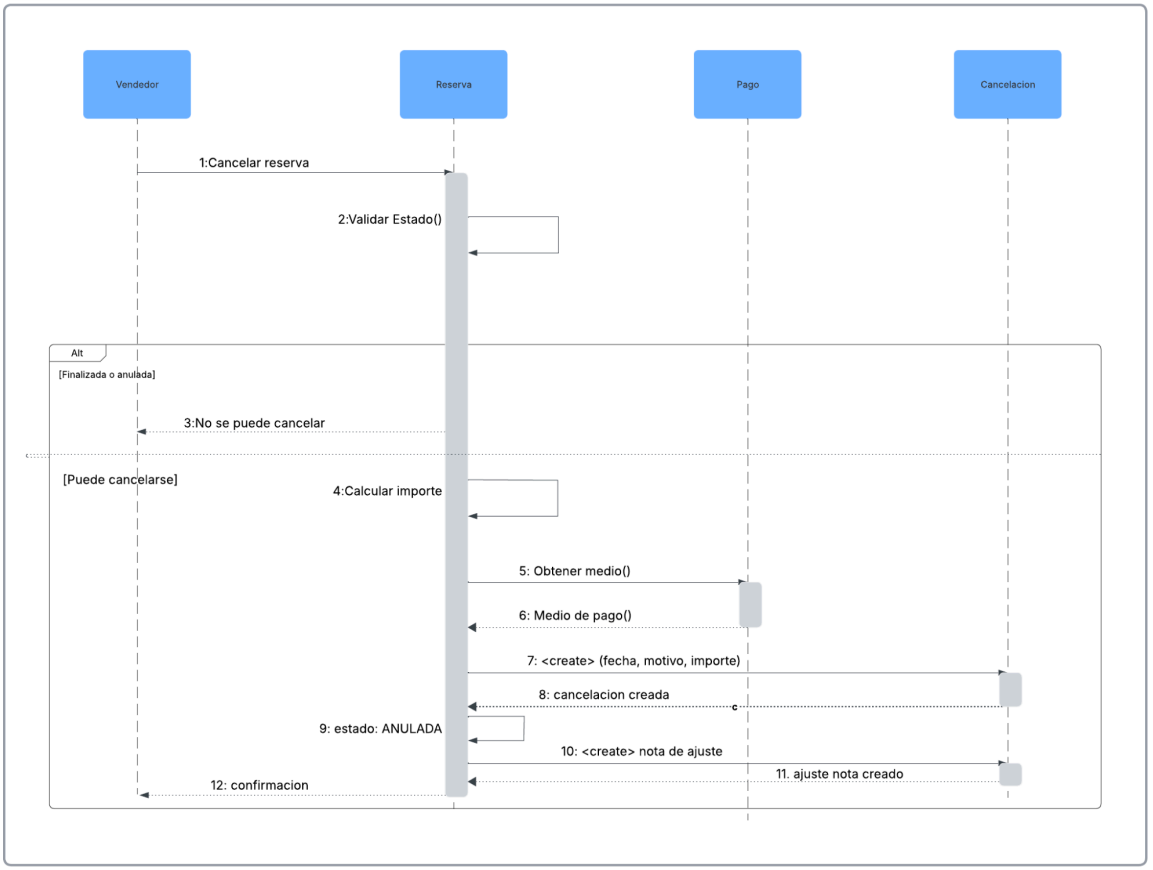
Diagramas de secuencia

Consultar salida



- El vendedor le pide a la agencia el listado de salidas
- La agencia busca ese listado y le pide obtenerDatos()
- La agencia le devuelve al vendedor el detalle: fecha, estado, guia y cupos

Cancelar reservas



- El vendedor pide cancelar la reserva y pasa el motivo
- La reserva valida su propio estado
- En el recuadro alt hay dos caminos; si esta FINALIZADA O ANULADA, devuelve un error y termina; si puede cancelarse, sigue con los mensaje 4 a 12
- En el camino normal , la reserva calcula el importe a devolver segun la politica (4), consulta el medio de pago de sus pagos (5,6) y crea la cancelacion (7)
- despues pasa a ANULADA (9) y crea la nota de ajuste (11)
- por ultimo confirma al vendedor(12)

Aplicación y justificación de GRASP

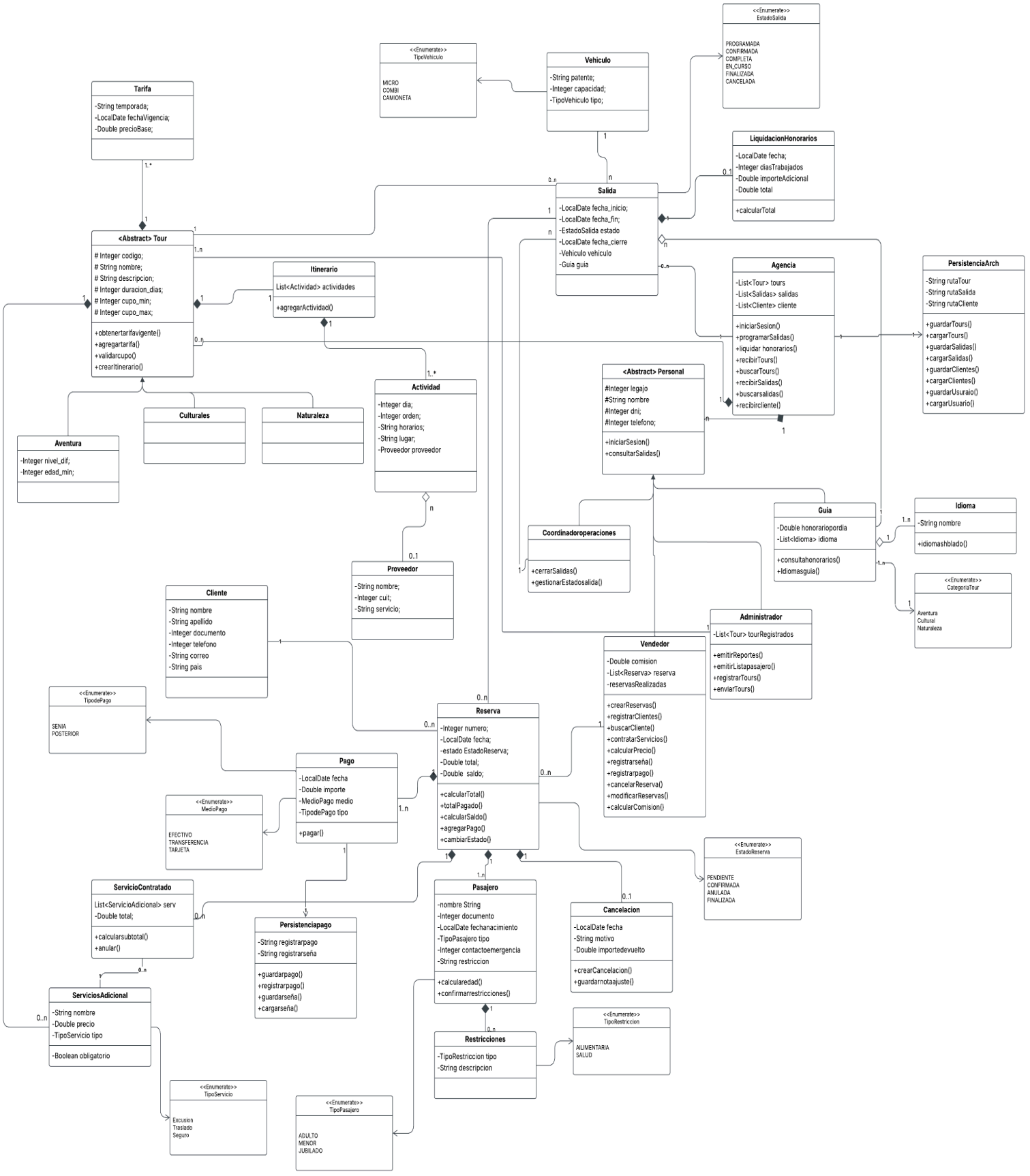
PRINCIPIOS SOLID

- Se detecto en la clase Agencia que hace diferentes responsabilidades programa salidas como también guarda salidas como una base de datos entonces decidí aplicar el principio solid SRP: una sola razón para cambiar y separar responsabilidades diferentes de la clase agencia con una nueva clase PersistenciaArch donde todo los archivos se guardan en esa clase y deja la clase agencia enfocada en su trabajo especifico el cual tendrá menor riesgos de efectos secundarios, una mayor cohesión interna para la clase
- Paso lo mismo con la clase pago en donde al registrarlospagos se guardan los archivos directamente en la clase pago entonces aplicando SRP: una sola razón para cambiar , se creo una nueva clase Persistenciapago en donde esos registros de pago puedan estar archivados en esa clase el cual seria mas fácil de leer y entender la información y mayor cohesión interna de la clase pago para que no se mezclen las actividades de cada clase

PATRONES GRASP

- En la clase tour se detecto que no hay un personal que registre los tours es por eso que decidí que la clase administrador sea el encargado de crear (registrar el tour) aplicando el patro de grasp creator como una ventaja de que deja claro quien crea el objeto, así que no hay dudas sobre donde va el “new”
- Anteriormente la clase agencia era la encargada de guardar toda la información y haciendo otras actividades independientes ahora se separo esa actividad con una clase persistencia el cual tiene guardado las informaciones y consultar en esa clase especifica las informaciones que se necesita aplicando el patron grasp information Expert en donde mejora la cohesión y reduce el acoplamiento y causa un efecto directo en la calidad del diseño
- Antes la clase coordinadoroperaciones hacia varias tareas hasta que se decidió en que se enfoca en una especifica actividad relacionada a su cargo logrando una alta cohesión con su propósito especifico aplicando el patron grasp high cohesión
- Se detecto un problema en el diseño que no hubo quien es el responsable de coordinar los eventos del sistema sin ejecutar la lógica del negocio, entonce se decidió en la clase Agencia funcione como controlador de fachada logrando separa la interfaz de usuario de la lógica del negocio
- Otro problema es que en la clase personal si cambiabas el comportamiento cambias el código entonces aplicando polymorphism podes cambiar ahora los comportamientos sin modificar el código
- La clase itinerario entre tour y actividad aplicando la indirection permite cambiar un lado sin afectar al otro , la clase intermedia desacopla dos clase que de otro modo dependerían directamente

Diagrama de clases final y cambios



La versión inicial del diagrama representaba la estructura del dominio (clases, atributos y relaciones), pero la mayoría de las clases no tenía operaciones. Eso mostraba qué información maneja el sistema, pero no quién es responsable de cada función. La versión actual completa el modelo asignando a cada clase las responsabilidades que le corresponden, de modo que cada requerimiento del sistema (consultar salidas, registrar tours, reservar, cobrar, cerrar salidas, liquidar honorarios) tenga una clase que lo ejecute y se pueda seguir en los diagramas de secuencia.

Resumen de cambios

Clase	Versión inicial	Versión actual	Motivo
Personal	Solo atributos comunes	Clase abstracta con iniciarSesion(), consultarSalidas() y consultarTours()	Concentra el comportamiento común del personal para no repetirlo en cada rol
Vendedor	Solo comision	Atributo reservasRealizadas y operaciones para crear y modificar reservas, registrar y buscar clientes, contratar servicios, calcular precio, registrar seña y pago, cancelar y calcular comisión	El vendedor es quien lleva adelante todo el ciclo de venta
Administrador	Cuatro reportes por separado (ocupación, ventas, tour, cancelaciones)	emitirReportes(), emitirListapasajero(), registrarTours(), enviarTours() y atributo tourRegistrados	Se unifican los reportes y se incorpora el alta de tours, que es responsabilidad del administrador
Coordinador	Solo CerrarSalidas()	Pasa a llamarse Coordinadoroperaciones y agrega gestionarEstadosalida()	Además de cerrar, gestiona el estado de cada salida
Agencia	Sin atributos y solo Liquidar honorarios()	Listas de tours, salidas y clientes, y operaciones para programar salidas, recibir y buscar tours, salidas y clientes, iniciar sesión y liquidar honorarios	Funciona como punto central que reúne la información del negocio y coordina las operaciones
EstadoSalida	PROGRAMADA, EN_CURSO, Cerrada	PROGRAMADA, CONFIRMADA, COMPLETA, EN_CURSO, FINALIZADA, CANCELADA	Refleja el ciclo de vida real: una salida se confirma al alcanzar el cupo mínimo, se completa al llegar al máximo y puede cancelarse

ServiciosAdicional	Sin tipo	Se agrega tipo (TipoServicio)	Permite clasificar el servicio como excursión, traslado o seguro
Pago	Sin operaciones	Se agrega pagar()	La clase tiene su propia operación para concretar el cobro de seña o pago posterior

Fundamentos de los cambios

Primero, se buscó que todas las operaciones del sistema tuvieran un responsable claro. En la versión inicial, por ejemplo, el vendedor figuraba solo con una comisión, aunque en la práctica es el actor que más operaciones realiza. Segundo, se agrupó el comportamiento repetido en la clase abstracta Personal, y se unificaron los cuatro reportes del administrador en una operación general. Tercero, se enriquecieron los estados de la salida para que el modelo represente el proceso completo, desde que se programa hasta que se finaliza o se cancela, y se pueda vincular con la regla de cupo mínimo y máximo del tour. Por último, se dotó a la Agencia de atributos y operaciones para que concentre los datos principales y pueda responder a consultas como la búsqueda de tours, salidas y clientes.

Como resultado, el diagrama de clases es coherente con los casos de uso y con los diagramas de secuencia, porque cada mensaje entre objetos corresponde a una operación definida en alguna clase.